Paradigmas de Programação

Programação Funcional: Lista 2

Parte A - Funções de alta ordem

1. Usando apenas as funções das bibliotecas `Prelude` e `Data.Char` e funções de alta ordem, implemente, de forma tácita, as funções listadas abaixo.

(a) `count :: String -> Int`, que recebe uma string e retorne o número de palavras que terminem com letra minúscula.

```
ghci> count "Um exemplO de Contagem de PalavraS"    -- 4
```

Solução:

`words` separa a string em palavras, `last` extrai o último caractere de cada palavra (nunca é vazia) e `isLower` testa se ele é minúsculo. `filter` mantém as palavras que passam no teste e `length` as conta.

```haskell
-- 1.a count
count :: String -> Int
count = length . filter (isLower . last) . words
```

Soluções alternativas: a primeira usa `map` para obter uma lista de `Bool` (um por palavra) e conta os verdadeiros com `filter id`; a segunda extrai o último caractere de cada palavra com `map last` e filtra os caracteres minúsculos com `isLower`.

```haskell
count_alt1 :: String -> Int
count_alt1 = length . filter id . map (isLower . last) . words

count_alt2 :: String -> Int
count_alt2 = length . filter isLower . map last . words
```

(b) `inverses :: [Int] -> [Double]`, que recebe uma lista de inteiros e retorne uma lista com os inversos multiplicativos dos elementos não-nulos. Ex

```
ghci> inverses [0, 1, 0, 2, 0, 4]                    -- [1.0, 0.5, 0.25]
```

Solução:

`filter (/= 0)` descarta os zeros; em seguida, `fromIntegral` converte cada `Int` em `Double` e `(1 /)` calcula o inverso.

```haskell
-- 1.b inverses
inverses :: [Int] -> [Double]
inverses = map ((1 /) . fromIntegral) . filter (/= 0)
```

Soluções alternativas: `recip x` é o mesmo que `1 / x`.

```haskell
inverses_alt1 :: [Int] -> [Double]
inverses_alt1 = map (recip . fromIntegral) . filter (/= 0)
```

(c) `odds :: Int -> Int`, que recebe um inteiro e returne o número de dígitos ímpares deste número.

```
ghci> odds 12345                                    -- 3
```

Solução:

`abs` descarta o sinal, `show` converte o número em uma string de dígitos e `digitToInt` converte cada caractere de volta em inteiro, que é testado com `odd`.

```haskell
-- 1.c odds
odds :: Int -> Int
odds = length . filter (odd . digitToInt) . show . abs
```

Soluções alternativas: a primeira versão converte cada caractere em inteiro com `read [c]`; a segunda só testa se o caractere pertence a `"13579"`.

```haskell
odds_alt1 :: Int -> Int
odds_alt1 = length . filter odd . map (\c -> read [c] :: Int) . show . abs

odds_alt2 :: Int -> Int
odds_alt2 = length . filter (`elem` "13579") . show . abs
```

(d) `palindromes :: String -> Int`, que recebe uma string e retorna o número de palavras que são palíndromos. Ex.:

```
ghci> palindromes "Aia mussum ou mirim"             -- 2
```

Solução:

Para funções, `f <*> g` aplicada a um argumento w resulta em `f w (g w)`. Logo, `(==) <*> reverse` aplicada a w resulta em `w == reverse w`, isto é, testa se uma palavra é igual à sua inversa.

A comparação diferencia maiúsculas de minúsculas, como no exemplo do enunciado: `"Aia"` não é contada, e o resultado é 2 ("mussum" e "mirim").

```haskell
-- 1.d palindromes
palindromes :: String -> Int
palindromes = length . filter ((==) <*> reverse) . words
```

Soluções alternativas: a gente sempre tem que associar cada palavra o par formado por ela e pela sua inversa, e testar se as duas componentes são iguais.

Com algo como `uncurry (==)`, a primeira usa função anônima; a segunda `(,) <*> reverse`, que aplicada a w resulta em `(w, reverse w)`; a terceira `liftA2 (,) id reverse`.

```haskell
palindromes_alt1 :: String -> Int
palindromes_alt1 = length . filter (uncurry (==)) . map (\s -> (reverse s, s)) . words

palindromes_alt2 :: String -> Int
palindromes_alt2 = length . filter (uncurry (==)) . map ((,) <*> reverse) . words

palindromes_alt3 :: String -> Int
palindromes_alt3 = length . filter (uncurry (==)) . map (liftA2 (,) id reverse) . words
```

(e) `isOctNumber :: Int -> Bool`, que recebe um inteiro `x` e retorna verdadeiro se x representa um número em

base octal. Ex.:

```
ghci> isOctNumber 1203577                    -- True
```

Solução:

Um número está escrito em base octal se todos os seus dígitos estão em `"01234567"`. Assim, `show . abs` produz a string dos dígitos e `all (flip elem "01234567")` verifica que todos pertencem ao conjunto.

```haskell
-- 1.e isOctNumber
isOctNumber :: Int -> Bool
isOctNumber = all (flip elem "01234567") . show . abs
```

Soluções alternativas: a primeira versão define a função auxiliar `within` (pertinência em lista, com os argumentos invertidos) e a usa no lugar de `flip elem`; a segunda usa o fato de que os caracteres `'0'` a `'9'` são consecutivos e ordenados, de modo que um dígito é octal se, e somente se, é menor que `'8'`.

```haskell
within :: Eq a => [a] -> a -> Bool
within = flip elem

isOctNumber_alt1 :: Int -> Bool
isOctNumber_alt1 = all (within "01234567") . show . abs

isOctNumber_alt2 :: Int -> Bool
isOctNumber_alt2 = all (< '8') . show . abs
```

2. Usando uma única dobra à esquerda, implemente as funções abaixo. A implementação deve ser feita em uma única linha.

(a) `eval :: [Int] -> Int -> Int`, que recebe uma lista de inteiros `as` e um inteiro `x` e retorna o valor de $p(x)$, onde

$$p(x) = a_0 + a_1 x + a_2 x^2 + \ldots + a_N x^N$$

Observação: a lista `as` contém os coeficientes do polinômio do maior índice para o menor índice. Ex.:

```
-- Avalia o polinômio p(x) = x^2 - 5x + 6 no ponto x = 2
ghci> eval [1 -5 6] 2                        -- 0
```

Dica: utilize o algoritmo de Horner.

Solução:

O algoritmo de Horner reescreve $p(x) = (\ldots((a_N x + a_{N-1})x + a_{N-2})x + \ldots)x + a_0$. Como a lista já vem do maior para o menor índice, basta percorrê-la da esquerda para a direita com acumulador inicial 0, fazendo acc $\leftarrow$ acc $\cdot\, x + a$ a cada coeficiente a.

```haskell
-- 2.a eval
eval :: [Int] -> Int -> Int
eval as x = foldl (\acc a -> acc * x + a) 0 as
```

Exemplo ($p(x) = x^2 - 5x + 6$, $x = 2$): $((0 \cdot 2 + 1) \cdot 2 - 5) \cdot 2 + 6 = 0$.

Soluções alternativas: `(+) . (* x)` é a função `\acc a -> acc * x + a`, escrita de forma tácita: primeiro multiplica o acumulador por x e depois soma o coeficiente.

```haskell
eval_alt1 :: [Int] -> Int -> Int
eval_alt1 as x = foldl ((+) . (* x)) 0 as
```

(b) `k_factorial :: Int -> Int -> Int`, que recebe os inteiros `n` e `k` e retorna o k-fatorial $n!_{(k)}$ de n, onde

$$n!_{(k)} = \begin{cases} n \cdot (n-k)!_{(k)}, & \text{se } n > 0; \\ 1, & \text{se } -k < n \leq 0; \\ 0, & \text{caso contrário} \end{cases}$$

Ex.:

```
ghci> k_factorial 6 2                                -- 48
```

Solução:

Seja $k > 0$. Se $n \leq -k$, o resultado é 0, o que é obtido com a lista `[0]`. Caso contrário, o resultado é o produto de n, $n-k$, $n-2k$, ... enquanto os termos forem positivos: ao chegar a um termo $m \leq 0$, necessariamente $-k < m \leq 0$, e o caso base vale 1. Essa sequência é gerada por `iterate (subtract k) n` e cortada por `takeWhile (> 0)`; a dobra `foldl (*) 1` calcula o produto.

```haskell
-- 2.b k_factorial
k_factorial :: Int -> Int -> Int
k_factorial n k = foldl (*) 1 (if n <= -k then [0] else takeWhile (>0) (iterate (subtract k) n))
```

Exemplo: `k_factorial 6 2` $= 6 \cdot 4 \cdot 2 = 48$.

Soluções alternativas: em vez de `if`, acrescenta-se um elemento 0 à lista com a compreensão de listas `[0 | n <= -k]`, que é a lista `[0]` se $n \leq -k$ e a lista vazia caso contrário. Quando $n \leq -k$, o `takeWhile` resulta em lista vazia e o produto final é 0.

```haskell
k_factorial_alt1 :: Int -> Int -> Int
k_factorial_alt1 n k =
    foldl (*) 1 $ takeWhile (> 0) (iterate (subtract k) n) ++ [0 | n <= -k]
```

(c) `mean :: [Int] -> Double`, que recebe uma lista de inteiros `xs` e retorna a média aritmética destes números. Ex.:

```
ghci> mean [1..4]                                    -- 2.5
```

Solução:

Como só é permitida uma dobra, a soma e a quantidade de elementos são acumuladas simultaneamente em um par `(soma, quantidade)`. Ao final, `uncurry (/)` divide a primeira componente pela segunda. Para a lista vazia, o resultado é `NaN` $(0/0)$.

```haskell
-- 2.c mean
mean :: [Int] -> Double
mean = uncurry (/) . foldl (\(s, n) x -> (s + fromIntegral x, n + 1)) (0, 0)
```

Soluções alternativas: `f <*> g` aplicada a uma lista xs resulta em `f xs (g xs)`; assim, o termo divide a soma de xs pelo seu comprimento. O resultado é o mesmo, mas esta versão percorre a lista duas vezes: uma pela dobra e outra por `length`, por isso ela não segue estritamente a restrição de usar uma única dobra.

```haskell
mean_alt1 :: [Int] -> Double
mean_alt1 = (/) . fromIntegral . foldl (+) 0 <*> fromIntegral . length
```

(d) `arithmetic_progression_sum :: Int -> Int -> Int -> Int` que recebe o número de termos `n`, o primeiro termo `a` e a razão `q` de uma progressão aritmética e retorne a soma dos primeiros n termos desta progressão. Ex.:

```
ghci> arithmetic_progression_sum 10 1 2          -- 100
```

Solução:

`iterate (+ q) a` gera a progressão infinita $a, a+q, a+2q, \ldots$; `take n` guarda os n primeiros termos e a dobra `foldl (+) 0` os soma.

```haskell
-- 2.d arithmetic_progression_sum
arithmetic_progression_sum :: Int -> Int -> Int -> Int
arithmetic_progression_sum n a q = foldl (+) 0 . take n $ iterate (+ q) a
```

Exemplo: $1 + 3 + 5 + \cdots + 19 = 100$.

Soluções alternativas: a primeira versão escreve `iterate` com `flip`; a segunda dobra sobre os índices $0, 1, \ldots, n-1$, somando o termo $a + i\,q$ ao acumulador.

```haskell
arithmetic_progression_sum_alt1 :: Int -> Int -> Int -> Int
arithmetic_progression_sum_alt1 n a q = foldl (+) 0 . take n $ flip iterate a ((+) q)

arithmetic_progression_sum_alt2 :: Int -> Int -> Int -> Int
arithmetic_progression_sum_alt2 n a q = foldl (\acc i -> acc + a + i * q) 0 [0 .. n - 1]
```

(e) `exp_approx :: Double -> Double`, que recebe um número `x` e retorna a aproximação da exponencial por série de potências até o 11° termo, isto é,

$$e^x \approx 1 + \frac{x}{1!} + \frac{x^2}{2!} + \ldots + \frac{x^{10}}{10!}$$

Ex.:

```
ghci> exp_approx 2                          -- 7.3889947089947089
```

Solução:

O termo de índice n é $x^n/n!$, para $n = 0, 1, \ldots, 10$ (onze termos). Esses termos são calculados com `map` e somados com uma única dobra à esquerda.

```haskell
-- 2.e exp_approx
exp_approx :: Double -> Double
exp_approx x = foldl (+) 0 $ map (\n -> x ^ n / fromIntegral (product [1..n])) [0..10 :: Int]
```

Soluções alternativas: a primeira versão emparelha x com os expoentes por meio de `zip`; como o `zip` é truncado no menor operando, só são usados onze termos. A segunda usa o método de Horner: $e^x \approx 1 + x\big(1 + \frac{x}{2}\big(1 + \frac{x}{3}(\ldots(1 + \frac{x}{10}))\big)\big)$, calculado por uma dobra à esquerda com $n = 10, 9, \ldots, 1$ e acc $\leftarrow 1 + \text{acc} \cdot x/n$.

```haskell
exp_approx_alt1 :: Double -> Double
exp_approx_alt1 x =
    foldl (+) 0 $ map (\(a, b) -> (a ^ b) / fromIntegral (product [1 .. b])) $
        zip (replicate 11 x) [0 .. 10 :: Int]

exp_approx_alt2 :: Double -> Double
exp_approx_alt2 x = foldl (\acc n -> 1 + acc * x / n) 1 [10, 9 .. 1]
```

Parte B - I/O

3. Resolva, no Beecrowd, o exercício indicado em Haskell. Para formatar a saída, use a função `printf` do módulo `Text.Printf`.

(a) BEE 1007 – Diferença

Solução:

```haskell
import Text.Printf (printf)

main :: IO ()
main = do
    [a, b, c, d] <- map read . words <$> getContents :: IO [Int]
    printf "DIFERENCA = %d\n" (a * b - c * d)
```

(b) BEE 1008 – Salário

Solução:

```haskell
import Text.Printf (printf)

main :: IO ()
main = do
    [number, hours, rate] <- words <$> getContents
    let salary = read hours * read rate :: Double
    printf "NUMBER = %d\nSALARY = U$ %.2f\n" (read number :: Int) salary
```

(c) BEE 1011 – Esfera

Solução:

```haskell
import Text.Printf (printf)

main :: IO ()
main = do
    r <- readLn :: IO Double
    printf "VOLUME = %.3f\n" (4 / 3 * 3.14159 * r ^ (3 :: Int))
```

(d) BEE 1013 – O Maior

Solução:

```haskell
import Text.Printf (printf)

main :: IO ()
main = do
    values <- map read . words <$> getContents :: IO [Int]
    printf "%d eh o maior\n" (maximum values)
```

(e) BEE 1014 – Consumo

Solução:

```haskell
import Text.Printf (printf)

main :: IO ()
main = do
    [x, y] <- words <$> getContents
    printf "%.3f km/l\n" (read x / read y :: Double)
```